

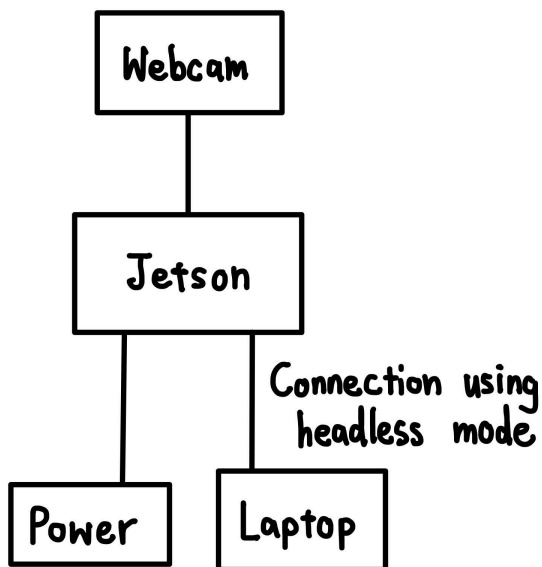
# Project Facial Recognition

## Overview

Project Facial Recognition is my first personal project using Nvidia Jetson Nano. To build a good foundation, I started with recognizing facial features in frames, a basic function that will also be utilized in the upcoming projects.

This project implements a real-time edge detection system using a USB webcam and an Nvidia Jetson Nano. Frames containing our faces are first captured by the webcam. Each pixel will be converted to grayscale and processed using the five stages of the Canny edge detection algorithm (Ding & Goshtasby, 2001). Regions with rapid density change correspond to the boundaries of our object. This measurement provides a lightweight method for detecting visual structures and motions without performing heavy calculations for full object recognition and without the need for high-end machines.

Using the Canny algorithm, this approach forms a foundation; base for more advanced computer-vision tasks such as face-tracking, biomedical monitoring, and autonomous systems.



**Project's structure:** The system consists of a webcam connected to a Jetson device for image processing. The Jetson is powered independently and controlled through a laptop using headless mode, allowing the project to run without a separate monitor or keyboard.

## Methods

This part will dive deeper into the methods and concepts used for this project.

### OpenCV

OpenCV (Open Source Computer Vision Library) is used to turn images into analyzable data. Once a frame is received from the webcam, it is viewed as a numerical image by OpenCV, and it manages all operations for the image from that point onward.

Besides the Canny edge detection algorithm in OpenCV, OpenCV also helps with

- preprocessing: ensuring grayscale (each image only stores brightness, and not colors)
- display: visualizing raw and processed images to verify and debug
- measure: Counting edge pixels, comparing frames over time, and extracting structural features

## Canny edge detection algorithm

This algorithm consists of 5 stages:

1. Noise Reduction: The Gaussian blur slightly removes tiny fluctuations so the algorithm doesn't mistake noise for structures.
2. Gradient Calculation: Measures how fast brightness changes in the horizontal and vertical direction of every pixel, which estimates the derivative of the image mathematically. Large gradient magnitude means something important is there, like an object boundary.
3. Non-Maximum Suppression: This algorithm keeps only the strongest pixels along the direction of the gradient and suppress its neighbors. This step turns blurry bands into crisp lines.
4. Double Thresholding: Pixels with strong gradients are marked as "edges"; pixels with very weak gradients are discarded. Pixels in between are marked as "maybe".
5. Edge Tracking by Hysteresis: This step decides the fate of the "maybe" pixels. A weak pixel is kept only if it connects to strong edges. Isolated weak pixels vanish.

## References

Ding, L., & Goshtasby, A. (2001). On the Canny edge detector. *Pattern recognition*, 34(3), 721-725.  
Sahir, S. (2019). Canny Edge Detection Step by Step in Python — Computer Vision.

## Code

Refer to [https://github.com/Mitch-Jimmy/Project\\_Facial\\_Recognition/blob/main/Main](https://github.com/Mitch-Jimmy/Project_Facial_Recognition/blob/main/Main) for the code.

## Results

```
Motion: 227 Objects: 1 FacePresent: 1 Faces: 0 EyesOpen: 0 Eyes: 0
Motion: 1792 Objects: 1 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
FPS: 14.55
Motion: 1018 Objects: 1 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
EVENT: motion stopped
Motion: 152 Objects: 0 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
Motion: 16 Objects: 0 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
FPS: 14.62
Motion: 1 Objects: 0 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
Motion: 0 Objects: 0 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
Motion: 0 Objects: 0 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
FPS: 14.68
EVENT: motion started
Motion: 1123 Objects: 1 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
Motion: 2290 Objects: 1 FacePresent: 0 Faces: 1 EyesOpen: 0 Eyes: 2
Motion: 2856 Objects: 1 FacePresent: 0 Faces: 1 EyesOpen: 1 Eyes: 2
FPS: 14.60
Motion: 2469 Objects: 1 FacePresent: 1 Faces: 1 EyesOpen: 1 Eyes: 2
Motion: 2254 Objects: 1 FacePresent: 1 Faces: 1 EyesOpen: 1 Eyes: 2
Motion: 2353 Objects: 1 FacePresent: 1 Faces: 1 EyesOpen: 1 Eyes: 2
FPS: 14.65
Motion: 2305 Objects: 1 FacePresent: 1 Faces: 1 EyesOpen: 1 Eyes: 2
Motion: 1604 Objects: 1 FacePresent: 1 Faces: 1 EyesOpen: 1 Eyes: 2
Motion: 2008 Objects: 1 FacePresent: 1 Faces: 1 EyesOpen: 1 Eyes: 2
FPS: 14.69
Motion: 1289 Objects: 1 FacePresent: 1 Faces: 1 EyesOpen: 1 Eyes: 2
Motion: 2204 Objects: 1 FacePresent: 1 Faces: 1 EyesOpen: 1 Eyes: 2
Motion: 2376 Objects: 1 FacePresent: 1 Faces: 1 EyesOpen: 1 Eyes: 2
FPS: 14.72
Motion: 2047 Objects: 1 FacePresent: 1 Faces: 0 EyesOpen: 0 Eyes: 0
Motion: 2051 Objects: 1 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
Motion: 1550 Objects: 1 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
FPS: 14.77
Motion: 423 Objects: 0 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
EVENT: motion stopped
Motion: 45 Objects: 0 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
Motion: 4 Objects: 0 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
FPS: 14.78
Motion: 512 Objects: 0 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
EVENT: motion started
Motion: 284 Objects: 0 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
EVENT: motion stopped
Motion: 364 Objects: 0 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
FPS: 14.80
EVENT: motion started
Motion: 880 Objects: 1 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
Motion: 1200 Objects: 1 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
Motion: 2012 Objects: 1 FacePresent: 0 Faces: 0 EyesOpen: 0 Eyes: 0
FPS: 14.81
^CStopping...
FPS: 14.8158186/1118101
```

Face entered the frame

Face and eyes are detected

Object still in frame but no face and eyes detected

Object (face) left the frame

**The program's real-time facial detection output results in the terminal:** The system logs motion values, object presence, face detection, and eye detection for each frame. When a person enters the camera view, the detected face and eyes are recorded, and when the person leaves the frame, the values return to zero.